Evidencia de Aprendizaje 2: Optimizando la productividad en el mundo del software

Control de versiones con GitHub y flujo DevOps con GitHub Actions para el web scraping del catálogo de tenis para hombre de Tiendas Branchos

Kevin Daniel Sepúlveda Giraldo
Jessica Serna Vargas
Grupo 9

Institución Universitaria Digital de Antioquia
Ingeniería de Software y Datos
Programación para Análisis de Datos — PREICA2602B020048
Docente: Andrés Felipe Palacio
10 de noviembre de 2026

Introducción

En la Evidencia de Aprendizaje 1 extrajimos el catálogo de tenis para hombre de Tiendas Branchos con BeautifulSoup, Selenium y Scrapy, y lo analizamos siguiendo las tres primeras fases de CRISP-DM. El resultado fue un dataset de 593 referencias y un cuaderno de Google Colab que funcionaba, pero que tenía las limitaciones típicas de un trabajo hecho en una sola sesión: todo el código estaba en celdas, no había forma de saber si un cambio dañaba algo y, para actualizar los precios, había que volver a ejecutar todo a mano.

En esta segunda evidencia tomamos ese mismo trabajo y lo llevamos a un flujo de desarrollo como el que usaría un equipo de software. Organizamos el código como un proyecto de Python, lo pusimos bajo control de versiones con Git y GitHub, trabajamos con ramas y pull requests, y configuramos tres flujos de GitHub Actions: integración continua con pruebas automáticas, un scraping programado que extrae el catálogo cada semana y valida que esté completo, y una entrega continua que publica un reporte en GitHub Pages cada vez que hay cambios en la rama principal.

El informe describe la página y el artículo que se retoman de la EA1, el tema de interés, los objetivos, la metodología de scraping y el flujo DevOps implementado, y los resultados obtenidos, incluyendo la comparación del catálogo entre el 19 de octubre y el 9 de noviembre de 2026.

Descripción de la página y del artículo analizado

La página: Tiendas Branchos

Tiendas Branchos (tiendasbranchos.com) es una cadena colombiana de calzado y ropa deportiva con tiendas físicas y venta en línea. Su tienda virtual está construida sobre la plataforma VTEX y vende marcas como Nike, Adidas, Puma, Reebok, Skechers, Fila y su marca propia Treme. Como se explicó en la EA1, la página es adecuada para scraping porque cada listado trae los productos en formato JSON-LD (estándar schema.org) y las tarjetas de producto muestran el precio actual y el precio anterior tachado. Además, la misma tienda fue la que se usó en el laboratorio de la semana 4, donde se mostró la extracción por JSON-LD y la sincronización del CSV con GitHub.

El artículo: tenis para hombre

Se mantiene la categoría completa de tenis para hombre (/hombres/tenis), que incluye tenis, guayos, teniguayos, botas y sandalias. En la EA1 la categoría tenía 593 referencias en 50 páginas; en la extracción del 9 de noviembre de 2026 la tienda reportó 560 referencias en 47 páginas de 12 productos.

Descripción del tema de interés

El tema sigue siendo el análisis de precios y descuentos del calzado deportivo en línea en Colombia, pero en esta unidad el interés se mueve de la pregunta "¿cómo están los precios hoy?" a "¿cómo cambian en el tiempo?". En la EA1 se concluyó que un descuento promedio de 37,5 % en casi todo el catálogo hacía pensar que el precio tachado era más una referencia comercial que una oferta real, pero con una sola extracción no era posible comprobarlo.

Para responder esa pregunta se necesita extraer el catálogo de forma periódica, guardar cada extracción sin sobrescribir las anteriores y tener la seguridad de que cada dataset está completo. Ese es el problema que resuelve un flujo DevOps: automatizar la ejecución, probar el código antes de integrarlo y publicar los resultados sin intervención manual. Por eso el tema técnico de esta evidencia es la gestión de versiones del proyecto de scraping en GitHub y su automatización con un pipeline de CI/CD.

Objetivos

Para el desarrollo de esta evidencia, asumimos el rol de integrantes de un equipo de desarrollo encargado de dar mantenimiento y continuidad al proyecto de scraping de Tiendas Branchos, realizado en la EA1. A partir de esta situación, nos planteamos los siguientes objetivos:

Objetivo general

Implementar un flujo de trabajo DevOps para el proyecto de web scraping de Tiendas Branchos, usando Git y GitHub para el control de versiones y GitHub Actions para la integración continua, la ejecución automática de la extracción y el despliegue continuo de un reporte de precios.

Objetivos específicos

1. Reorganizar el código de la EA1 como un paquete de Python con módulos separados para extracción, limpieza, análisis y reporte, de forma que cada parte se pueda probar por separado.
2. Gestionar el proyecto en un repositorio público de GitHub con una rama principal protegida por el proceso de revisión y ramas de funcionalidad (scraping, procesamiento y despliegue) integradas mediante pull requests.
3. Configurar un pipeline de integración continua que ejecute análisis de estilo y pruebas automatizadas en varias versiones de Python en cada push y pull request.
4. Automatizar la extracción semanal del catálogo con un control de calidad que impida publicar datasets incompletos.
5. Desplegar de forma continua un reporte con los indicadores del catálogo en GitHub Pages y documentar el flujo en el repositorio.

Metodología empleada

Marco metodológico

El proyecto sigue organizado con CRISP-DM. En la EA1 se trabajaron las fases de comprensión del negocio, comprensión de los datos y preparación de los datos. En esta evidencia esas fases quedan implementadas como código probado y se agrega la base de la fase de despliegue: el resultado del análisis se publica automáticamente y se actualiza cada semana. Sobre CRISP-DM se aplicaron las prácticas DevOps vistas en la Unidad 2: control de versiones, revisiones mediante pull requests, integración continua, entrega continua, gestión de la configuración y monitoreo de las ejecuciones.

Metodología de scraping

La extracción conserva el método 1 de la EA1 (requests + BeautifulSoup), porque fue el más completo: además del JSON-LD lee las tarjetas VTEX para obtener el precio tachado. Selenium y Scrapy cumplieron su papel de comparación en la EA1, pero para una ejecución automática en GitHub Actions no tiene sentido instalar un navegador completo ni mantener dos

implementaciones. Lo que sí se cambió fue la forma de manejar las fallas que encontramos en la EA1:

- **Reintentos por página:** si la tienda responde una página vacía o con error, el scraper la vuelve a pedir hasta tres veces con una espera creciente.
- **Segunda pasada:** en la EA1 se recuperaron las páginas perdidas uniendo BeautifulSoup y Scrapy. Ahora, al terminar el recorrido, el scraper vuelve a pedir solo las páginas que quedaron pendientes.
- **Validación contra el total de la tienda:** se compara el número de registros con el campo recordsFiltered que reporta VTEX. Si la cobertura es menor al 95 %, o la completitud de los campos clave es menor al 95 %, o hay duplicados o precios en cero, el script termina con error y el pipeline se detiene sin publicar.
- **Ética:** se revisa el robots.txt antes de extraer, se espera 1,5 segundos entre peticiones, se hace una sola petición a la vez, solo se toma información pública y la ejecución automática es semanal.

Estructura del proyecto

El código del cuaderno de la EA1 se separó en cinco módulos dentro de src/branchos y dos scripts que forman el pipeline:

Tabla 1

Módulos y scripts del proyecto

Archivo	Responsabilidad
config.py	Parámetros: categoría, pausa, reintentos, encabezados y columnas
scraper.py	Lectura del JSON-LD y las tarjetas VTEX, reintentos, segunda pasada y robots.txt
limpieza.py	Conversión de precios, tipo de calzado, descuento, duplicados y control de calidad
analisis.py	Indicadores del catálogo, resumen por marca, rangos de precio y comparación entre fechas
reporte.py	Sitio estático con gráficos de matplotlib y CSV descargable
scripts/extraer.py	Paso 1 del pipeline: scraping, limpieza, validación y guardado en data/
scripts/publicar.py	Paso 2 del pipeline: genera el sitio a partir de data/ y del histórico

Nota. Elaboración propia.

Control de versiones y flujo de ramas

El repositorio público es https://github.com/kevinsepulveda-iud/branchos-precios-devops. Siguiendo un flujo tipo GitHub Flow, la rama main solo recibe cambios por pull request y cada funcionalidad se desarrolla en su propia rama. Los commits siguen el formato tipo(alcance): descripción que se usó en el laboratorio de clase (feat, test, ci, data, docs, build).

Tabla 2

Ramas del repositorio y pull requests

Rama	Contenido	Commits
feature/scraping	Configuración, limpieza, scraper, pruebas con una página real guardada y workflow de CI	4
feature/procesamiento	Análisis, reporte, script de publicación y dataset del 9 de noviembre	3
feature/despliegue	Workflows de CD y de scraping programado, Dockerfile, README y cuaderno	4
fix/grafico-marcas	Corrección de la leyenda del gráfico por marca y nombres del grupo en el reporte	2

Nota. Cada rama se integró a main con un pull request (#1 a #4) después de que el CI terminara en verde.

Para trabajar desde Colab, como en el laboratorio de la semana 4, el cuaderno clona el repositorio. A diferencia del ejemplo de clase, el token de acceso no se escribe en el código: se lee desde los secretos de Colab (userdata.get("GITHUB_TOKEN")), para que no quede expuesto si se comparte el archivo.

Pruebas automatizadas

Escribimos 40 pruebas con pytest. Las del scraper usan un recorte de una página real de Branchos guardada en tests/fixtures, de modo que corren sin internet y no le hacen peticiones a la tienda en cada push; las fallas de red se simulan con una sesión falsa que devuelve respuestas vacías o con código 503. Varias pruebas salen directamente de problemas reales: la página que llega vacía, la extracción incompleta que casi entregamos en la EA1 y un nombre de producto que llegó con la entidad HTML " en esta extracción.

Pipeline de CI/CD con GitHub Actions

Tabla 3

Workflows de GitHub Actions

Workflow	Se ejecuta cuando	Qué hace
ci.yml	Push a main o a feature/*, y pull requests	Instala dependencias, revisa el estilo con ruff y corre pytest con cobertura en Python 3.10, 3.11 y 3.12 (matriz)
scraping-programado.yml	Lunes 6:00 a. m. (cron) o manual	Ejecuta extraer.py; si pasa el control de calidad hace commit del dataset en data/historico
cd.yml	Push a main o fin exitoso del scraping	Vuelve a correr las pruebas, genera el sitio con publicar.py y lo despliega en GitHub Pages

Nota. Los archivos están en .github/workflows. El Dockerfile permite ejecutar el mismo pipeline en cualquier máquina.

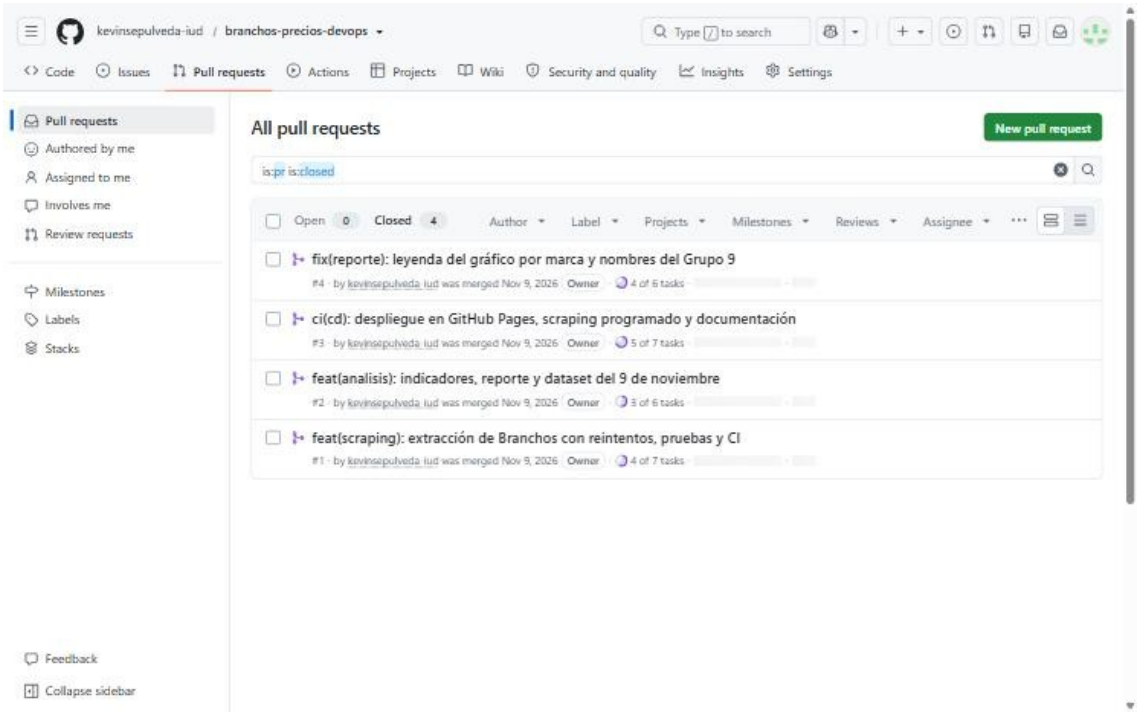
El flujo completo queda así: un cambio se desarrolla en una rama feature, el push dispara el CI, se abre un pull request hacia main y solo se hace merge con las pruebas en verde. El merge dispara el CD, que vuelve a probar y publica el reporte. Por otro lado, cada lunes el scraping programado actualiza los datos y su commit vuelve a disparar el CD, sin que nadie tenga que ejecutar nada.

Resultados

Repositorio y ejecución del pipeline

Figura 1

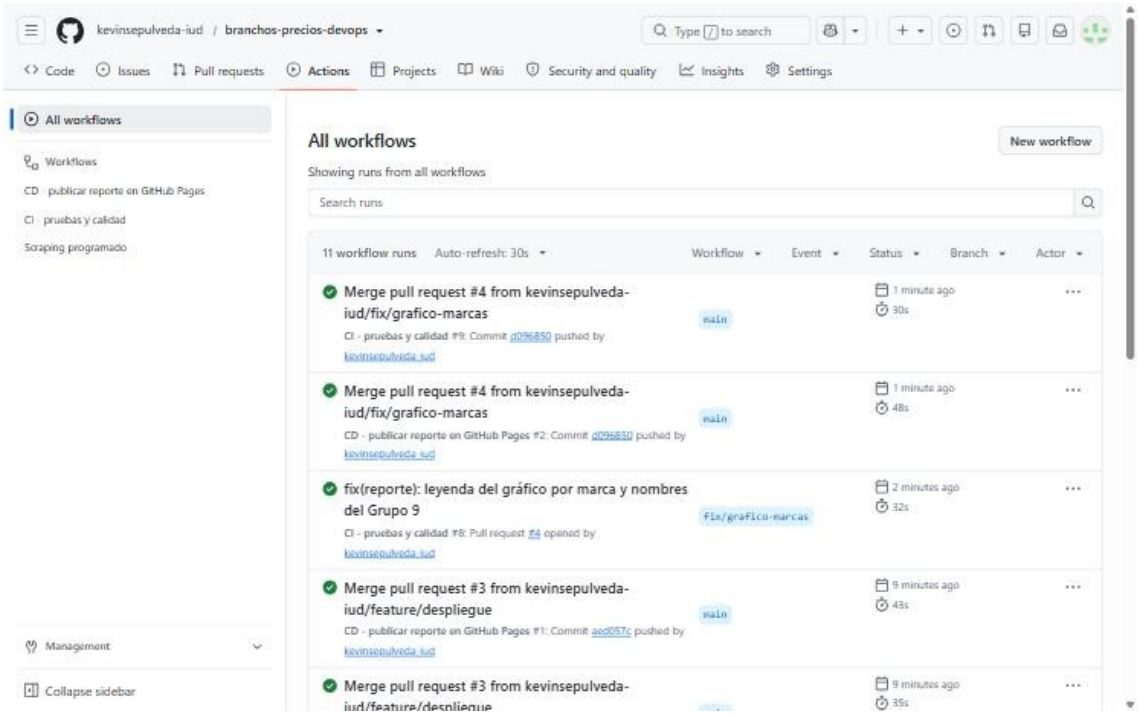
Pull requests integrados en el repositorio



Nota. Captura del repositorio kevinsepulveda-iud/branchos-precios-devops, 9 de noviembre de 2026. Los cuatro pull requests se integraron a main con el CI en verde.

Figura 2

Ejecuciones de los workflows en GitHub Actions



Nota. Captura de la pestaña Actions. Las 11 ejecuciones de los workflows de CI y CD terminaron con éxito.

Figura 3

Reporte publicado en GitHub Pages



Nota. Captura de https://kevinsepulveda-iud.github.io/branchos-precios-devops/, publicado automáticamente por el workflow cd.yml.

El repositorio quedó con 14 commits repartidos en la rama main y cuatro ramas, integradas con los pull requests #1 a #4 (Figura 1). Los tres primeros corresponden a las ramas de funcionalidad. El cuarto salió de revisar el reporte ya publicado: la leyenda del gráfico por marca tapaba dos barras y el reporte no mostraba a los dos integrantes del grupo, así que se corrigió en una rama fix y se integró por el mismo flujo. Cada pull request disparó el workflow de CI, y en todos los casos las 40 pruebas y la revisión de estilo pasaron en las tres versiones de Python; en total se ejecutaron 11 corridas de GitHub Actions, todas exitosas (Figura 2).

Al integrar el PR #3 se activó por primera vez el workflow de CD, que volvió a correr las pruebas, generó el sitio y lo desplegó en GitHub Pages en menos de un minuto; al integrar el PR #4 el sitio se actualizó solo con las correcciones (Figura 3). Desde ese momento, cualquier cambio en main actualiza el reporte sin que tengamos que ejecutar nada manualmente. La primera ejecución automática del scraping programado será el lunes 16 de noviembre de 2026.

Extracción del 9 de noviembre de 2026

La extracción recorrió las 47 páginas de la categoría en 204,8 segundos y obtuvo 548 de las 560 referencias que reporta la tienda (97,9 %). La página 28 respondió vacía en los tres intentos y también en la segunda pasada, el mismo comportamiento intermitente de la caché de la tienda que se encontró en la EA1. Como la cobertura supera el mínimo de 95 % y la completitud de nombre, marca, precio, precio anterior y URL fue de 100 %, el dataset pasó el control de calidad y quedó versionado en data/historico/tenis_2026-11-09.csv.

Comparación con la EA1

Tabla 4

Indicadores del catálogo de tenis para hombre en las dos extracciones

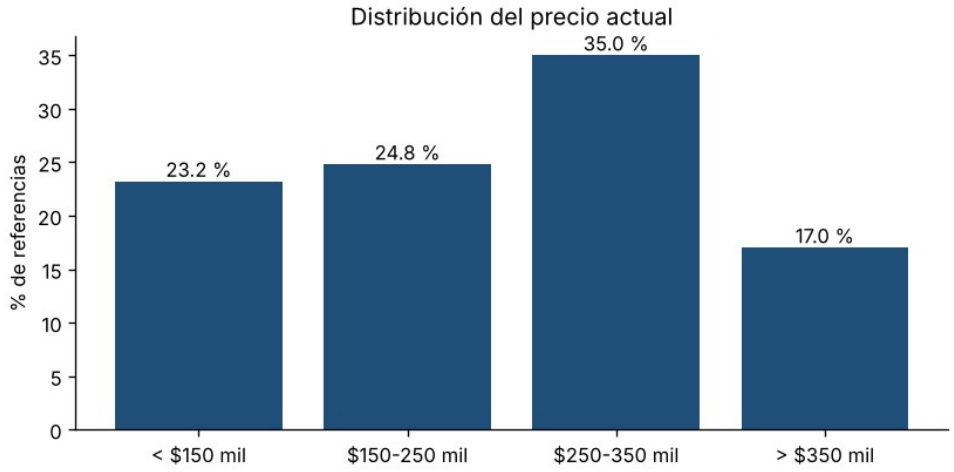
Indicador	EA1 (19 oct)	EA2 (9 nov)	Cambio
Referencias	593	548	−45
Marcas	7	7	0
Precio promedio	$ 220.234	$ 263.966	+$ 43.732
Precio mediano	$ 210.000	$ 259.000	+$ 49.000
Referencias con descuento	98,0 %	100 %	+2,0 pp
Descuento promedio	37,5 %	24,0 %	−13,5 pp
Descuento de 50 % o más	19,6 %	8,4 %	−11,2 pp
Tallas disponibles (promedio)	5,9	5,7	−0,2

Nota. Precios en pesos colombianos. pp: puntos porcentuales. La EA1 se toma del informe entregado; la EA2 del dataset versionado en el repositorio.

El cambio más claro está en los descuentos: en tres semanas el descuento promedio bajó 13,5 puntos y la proporción de productos con rebaja de 50 % o más pasó de casi uno de cada cinco a menos de uno de cada diez. Al mismo tiempo, el precio mediano subió $ 49.000. Todos los productos siguen mostrando un precio tachado, pero la distancia entre ese precio y el precio real se redujo. Esto refuerza la hipótesis de la EA1: el precio tachado se mantiene siempre y lo que la tienda mueve es el precio de venta. Con el histórico semanal se podrá confirmar si es un ciclo (por ejemplo, rebajas antes de temporadas como Black Friday) o un cambio de estrategia.

Figura 4

Distribución del precio actual — 9 de noviembre de 2026



Nota. Elaboración propia con datos extraídos de tiendasbranchos.com. Gráfico generado por el pipeline.

La oferta se desplazó hacia precios más altos: en la EA1 el 45,4 % de las referencias costaba entre $ 150.000 y $ 250.000; ahora el rango con más productos es el de $ 250.000 a $ 350.000, con el 35,0 %, y el 17,0 % supera los $ 350.000.

Tabla 5

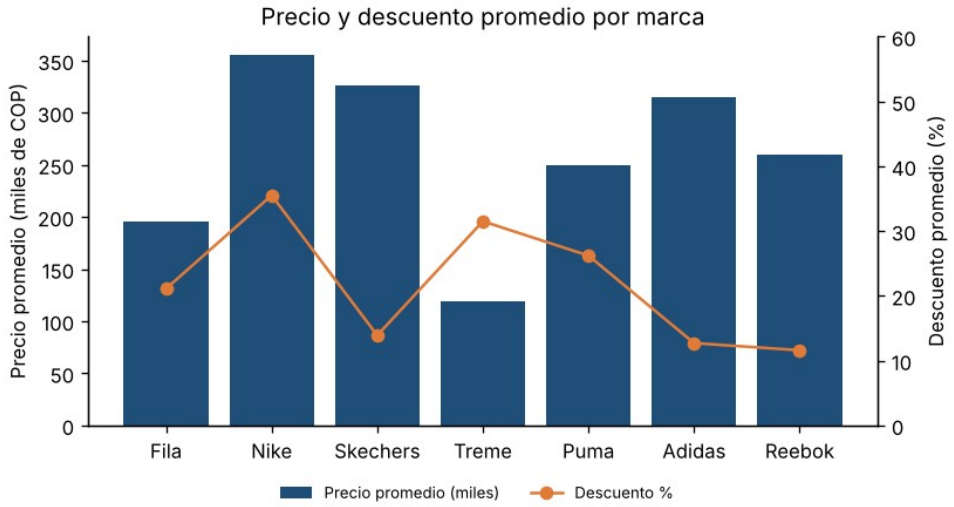
Referencias, precio y descuento promedio por marca — 9 de noviembre de 2026

Marca	Referencias	Precio promedio	Descuento promedio	Tallas promedio
Fila	133	$ 195.289	21,1 %	6,5
Nike	129	$ 355.261	35,5 %	4,6
Skechers	85	$ 326.298	13,9 %	6,1
Treme	66	$ 118.917	31,5 %	5,6
Puma	60	$ 249.408	26,2 %	6,0
Adidas	51	$ 315.278	12,7 %	5,1
Reebok	24	$ 259.333	11,6 %	6,5

Nota. Elaboración propia.

Figura 5

Precio y descuento promedio por marca

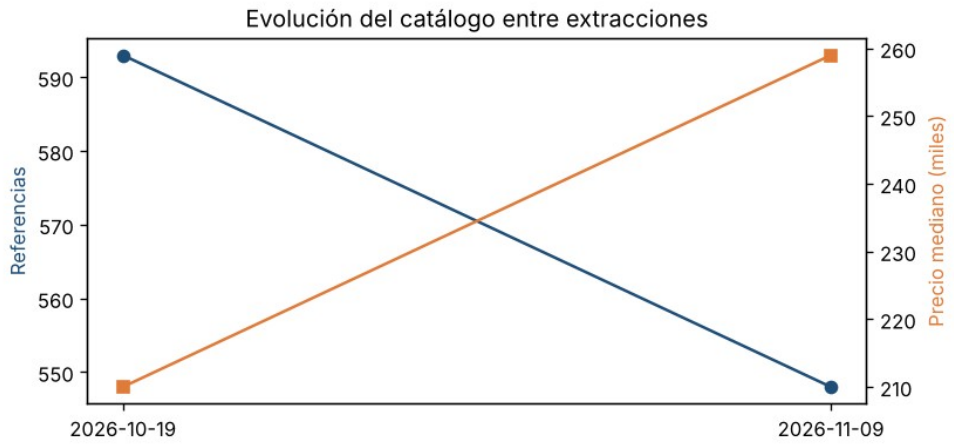


Nota. Elaboración propia con datos extraídos de tiendasbranchos.com.

El orden de las marcas se mantiene respecto a la EA1: Fila y Nike tienen la mayor cantidad de referencias, Nike es la más costosa y la que más descuenta, y Treme, la marca propia de la tienda, es la más económica. Las marcas que más redujeron sus descuentos fueron Adidas (de 31,5 % a 12,7 %), Skechers (de 30,4 % a 13,9 %) y Reebok (de 32,2 % a 11,6 %). Nike sigue siendo la marca con menos tallas disponibles por referencia (4,6). Por tipo de calzado, los guayos son los que tienen el descuento promedio más alto (34,2 %).

Figura 6

Evolución del catálogo entre extracciones



Nota. Este gráfico se actualiza automáticamente con cada extracción semanal.

Conclusiones

- Se cumplió el objetivo general: el proyecto de la EA1 quedó en un repositorio de GitHub con ramas, pull requests y tres workflows de GitHub Actions que prueban el código, extraen el catálogo cada semana y publican el reporte sin intervención manual.
- Separar el código del cuaderno en módulos fue lo que más trabajo nos implicó, pero es lo que hace posible la integración continua: con 40 pruebas que corren en tres versiones de Python, cualquier cambio que dañe la extracción se detecta antes de llegar a main.
- El control de calidad automático es la pieza más importante del pipeline. En la EA1 se detectó manualmente que se habían extraído solo 84 productos; ahora esa misma situación hace fallar el workflow y el reporte publicado no se reemplaza con datos incompletos.
- Trabajar con ramas nos obligó a dividir el proyecto en partes con un propósito claro (scraping, procesamiento y despliegue) y deja un historial que explica cómo se construyó, algo que en un cuaderno de Colab no queda registrado.
- En el negocio, la comparación entre las dos extracciones muestra que en tres semanas el descuento promedio bajó de 37,5 % a 24,0 % y el precio mediano subió de $ 210.000 a $ 259.000, mientras que el precio tachado se mantuvo. El scraping semanal permitirá confirmar si los descuentos son cíclicos.

- Como siguiente paso, para la EA3 el reporte estático se convertirá en una aplicación web desplegada en la nube, aprovechando el mismo pipeline de GitHub Actions.

Referencias

Chacon, S. y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. y Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

GitHub. (s. f.). *GitHub Actions documentation*. Recuperado el 9 de noviembre de 2026, de https://docs.github.com/es/actions

GitHub. (s. f.). *GitHub Pages documentation*. Recuperado el 9 de noviembre de 2026, de https://docs.github.com/es/pages

Humble, J. y Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Hüttermann, M. (2012). *DevOps for developers*. Apress.

Kim, G., Humble, J., Debois, P. y Willis, J. (2016). *The DevOps handbook*. IT Revolution Press.

Mitchell, R. (2018). *Web scraping with Python: Collecting more data from the modern web* (2.ª ed.). O'Reilly Media.

pytest-dev. (s. f.). *pytest documentation*. https://docs.pytest.org/

Tiendas Branchos. (2026). *Tenis para hombre*. Recuperado el 9 de noviembre de 2026, de https://www.tiendasbranchos.com/hombres/tenis

Anexo A. Implementación de la solución

El código fuente completo está en el repositorio https://github.com/kevinsepulveda-iud/branchos-precios-devops y se entrega también el cuaderno Sepulveda_Kevin_Optimizando_EA2.ipynb, que clona el repositorio, ejecuta las pruebas, muestra el resumen de la extracción, compara con la EA1 y genera el reporte. El reporte publicado está en https://kevinsepulveda-iud.github.io/branchos-precios-devops/.

Workflow de integración continua (.github/workflows/ci.yml), parte principal:

```
on:
  push:
    branches: [main, develop, "feature/**"]
  pull_request:
    branches: [main, develop]
jobs:
  pruebas:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: ${{ matrix.python-version }}, cache: pip }
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: pytest --cov=src/branchos --cov-report=term-missing
```

Segunda pasada del scraper (src/branchos/scraper.py):

```
# En la EA1 vi que las páginas que fallan suelen responder bien unos segundos
# después. En vez de usar un segundo método, vuelvo a pedir solo las pendientes.
recuperadas = []
for pagina in list(fallidas):
    time.sleep(pausa * 4)
    _, nuevos, _ = descargar_pagina(pagina, sesion)
    if nuevos:
        registros.extend(nuevos)
        fallidas.remove(pagina)
        recuperadas.append(pagina)
```

Control de calidad antes de publicar (src/branchos/limpieza.py):

```
if total_reportado:
    cobertura = len(df) / total_reportado * 100
    if cobertura < 95:
        problemas.append(f"Solo se extrajo el {cobertura:.1f} % de los
{total_reportado} productos")
```